CampusCare: A Firebase-Backed Mobile Application for Campus Incident Reporting

Student: Robert Tony Mitali Niyonkuru **Class:** January 25, BSE **Course:** Mobile Application Development
Submission date: 7 November 2026 **Repository:** https://github.com/RobertNiyonkuru98/campus_incident **Demonstration video:** https://drive.google.com/file/d/1Sjzg8yjzR8n9Rz8orTP4MHI2JZgVKfdz/view?usp=drive_link

---

Abstract

Campus facility faults are usually reported informally, which makes them difficult to track and easy to forget. CampusCare is a Flutter application that lets students submit incident reports with photographs and lets authorised staff move those reports through a defined resolution workflow. The application uses Firebase Authentication for identity, Cloud Firestore for persistent storage, and a third-party image host for photographs, with application state managed through the Cubit pattern. Access control is enforced by Firestore security rules rather than by the interface, so a student cannot change a report's status even if they bypass the app entirely. This report describes the architecture, the data model, the state management approach, the access-control design, the testing strategy, and the problems encountered during development.

---

1. Introduction

Students encounter broken facilities, water shortages, power outages, damaged furniture and sanitation problems throughout the academic term. Reporting these issues normally depends on word of mouth or on finding the correct staff member, so reports are frequently lost, duplicated, or submitted without enough detail to act on. The absence of a shared record also means a student has no way of knowing whether an issue has been acknowledged.

CampusCare addresses this by providing a single reporting channel with three properties: a structured report, a visible status, and a role-separated workflow. A student submits a report and can follow its progress. A staff member sees all open reports in one queue and records each status change. Every change is written to the report's timeline, so the history is auditable rather than reconstructed from memory.

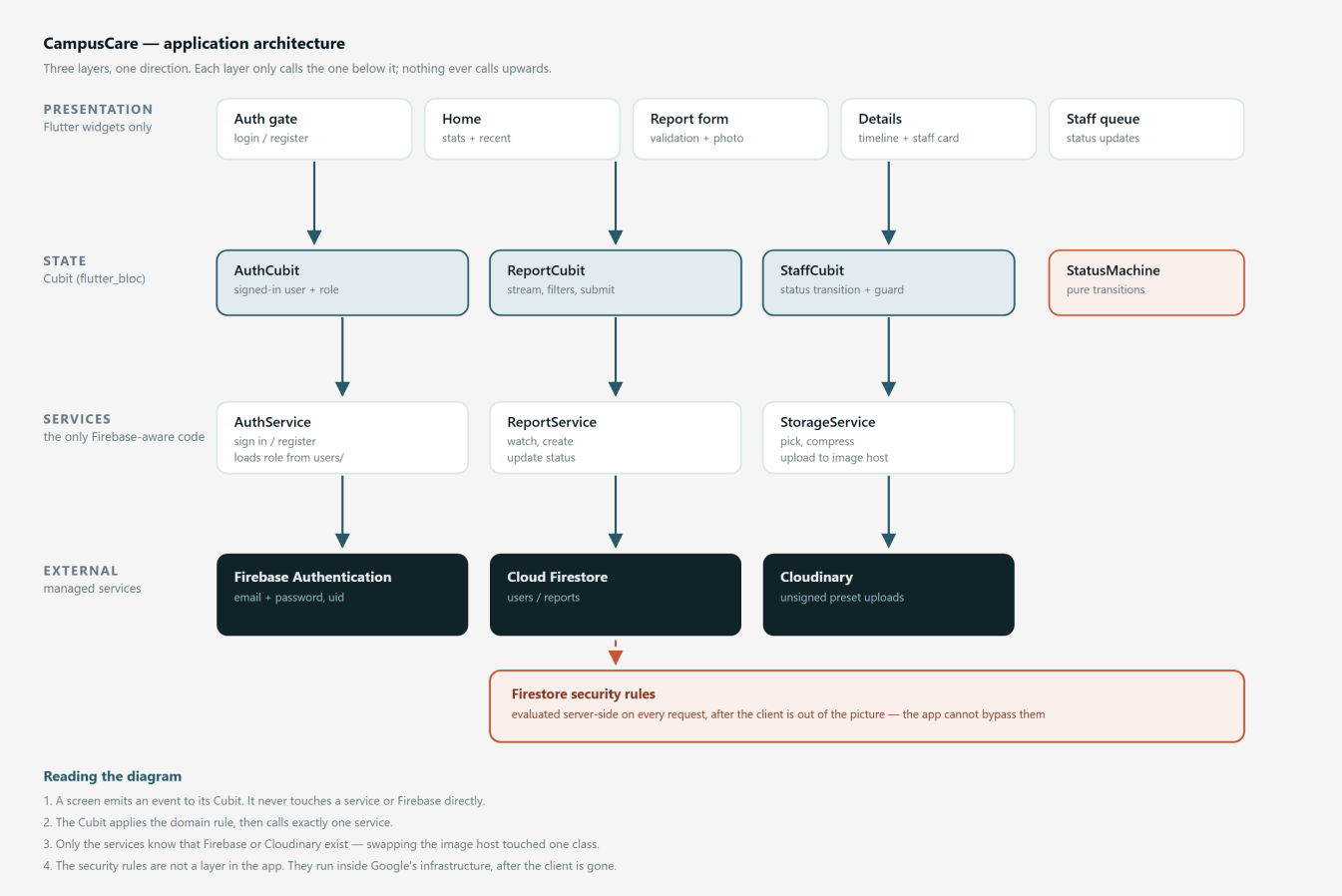
The scope of this implementation is deliberately limited. There is no administrative console, no push notification, and no assignment of reports to individual staff members. These are identified as future work in Section 9 rather than implemented partially.

---

2. Application architecture

The application is organised into three layers with a single direction of dependency.

Figure 1. Three-layer architecture with server-side rule enforcement.



The presentation layer contains Flutter widgets only. A screen never calls Firebase and never reads a database directly; it emits an event to a Cubit and renders whatever state the Cubit exposes. The state layer contains three cubits, one per bounded responsibility, plus a pure domain object that encodes the legal status transitions. The service layer contains three classes, and it is the only place in the codebase where `FirebaseAuth`, `FirebaseFirestore` or the image host are referenced.

The reason for this separation is testability rather than architectural decoration. Because the validation rules and the status machine are plain Dart with no Firebase dependency, they are covered by unit tests that run in under a second without an emulator or a network connection. Section 7 describes what is covered.

A second consequence concerns substitution. Photographs are hosted outside Firebase for reasons described in Section 8, and because storage sits behind a single `StorageService`, the change required edits to one class and one configuration file. No screen, Cubit, model or query was modified. That outcome is the practical argument for keeping external services behind an interface.

Dependencies are declared in `pubspec.yaml` and consist of `firebase_core`, `firebase_auth`, `cloud_firestore`, `flutter_bloc`, `image_picker`, `http` and `intl`. The `http` package handles the image upload as a multipart request, which avoided adding a vendor-specific SDK.

---

3. Firebase data structure

Firestore stores two top-level collections. Firestore has no join operation, and that constraint drives most of the modelling decisions below.

Table 1. The `users` collection

Field	Type	Purpose
`uid`	string	Mirrors the Authentication uid and is also the document id
`name`	string	Display name
`email`	string	Account email
`role`	string	`student` or `staff`. The access-control source of truth
`department`	string	Staff only, for example `Facilities`
`studentId`	string	Student registration number
`createdAt`	timestamp	Server timestamp, set on registration

Table 2. The `reports` collection

Field	Type	Purpose
`title`, `description`	string	The report itself
`category`	string	One of `facilities`, `water`, `electricity`, `safety`, `sanitation`
`location`	map	`{ label }`, kept as a map so building and room can be split later
`photoUrls`	array of string	Public HTTPS URLs from the image host
`status`	string	`submitted`, `inProgress` or `resolved`
`submittedBy`	map	`{ uid, name }`, denormalised
`createdAt`, `updatedAt`	timestamp	Server timestamps
`resolvedAt`	timestamp	Written when the report is resolved
`staffNotes`	string	The most recent staff note
`timeline`	array of map	`{ status, at, byUid, byName, note }`

Three decisions in this model are worth justifying.

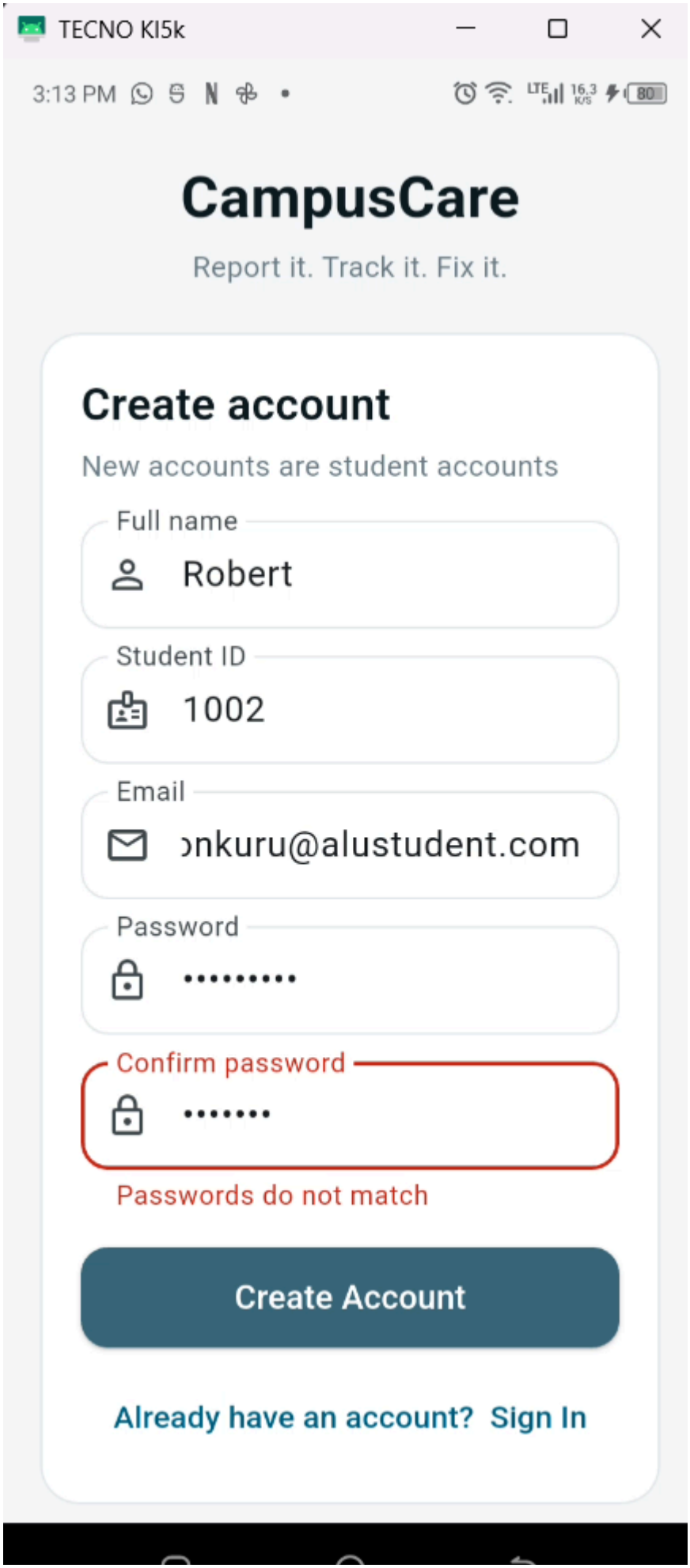
The `submittedBy` field is a map rather than a bare uid. A list screen that showed only a uid would need one additional document read per row to display the reporter's name, which produces an N+1 read pattern on the most frequently opened screen in the application. Denormalising the name makes the list a single query. The cost is that renaming a user does not update historical reports, which is acceptable because names change rarely and only staff can change them.

The `timeline` is an array of maps rather than a subcollection. It is always read in full when the report detail screen opens, and it is never queried independently, so a subcollection would add documents without adding capability. An array keeps the whole history in the document that is already being fetched.

Status and category filtering is performed on the client after a single ordered query. This keeps the application at one index-free query and makes filtering instantaneous, at the cost of holding every report in memory. For a single

campus that trade is reasonable; at a larger scale the filters would move into the query and require composite indexes. That limit is stated explicitly rather than hidden.

Figure 2. Client-side validation on the registration form.


TECNO KI5k
3:13 PM
CampusCare
Report it. Track it. Fix it.
Create account
New accounts are student accounts
Full name
Robert
Student ID
1002
Email
ɔnkuru@alustudent.com
Password
Confirm password
Passwords do not match
Create Account
Already have an account? Sign In



4. State management

State is managed with Cubit from the `flutter_bloc` package. Three cubits divide the application by responsibility.

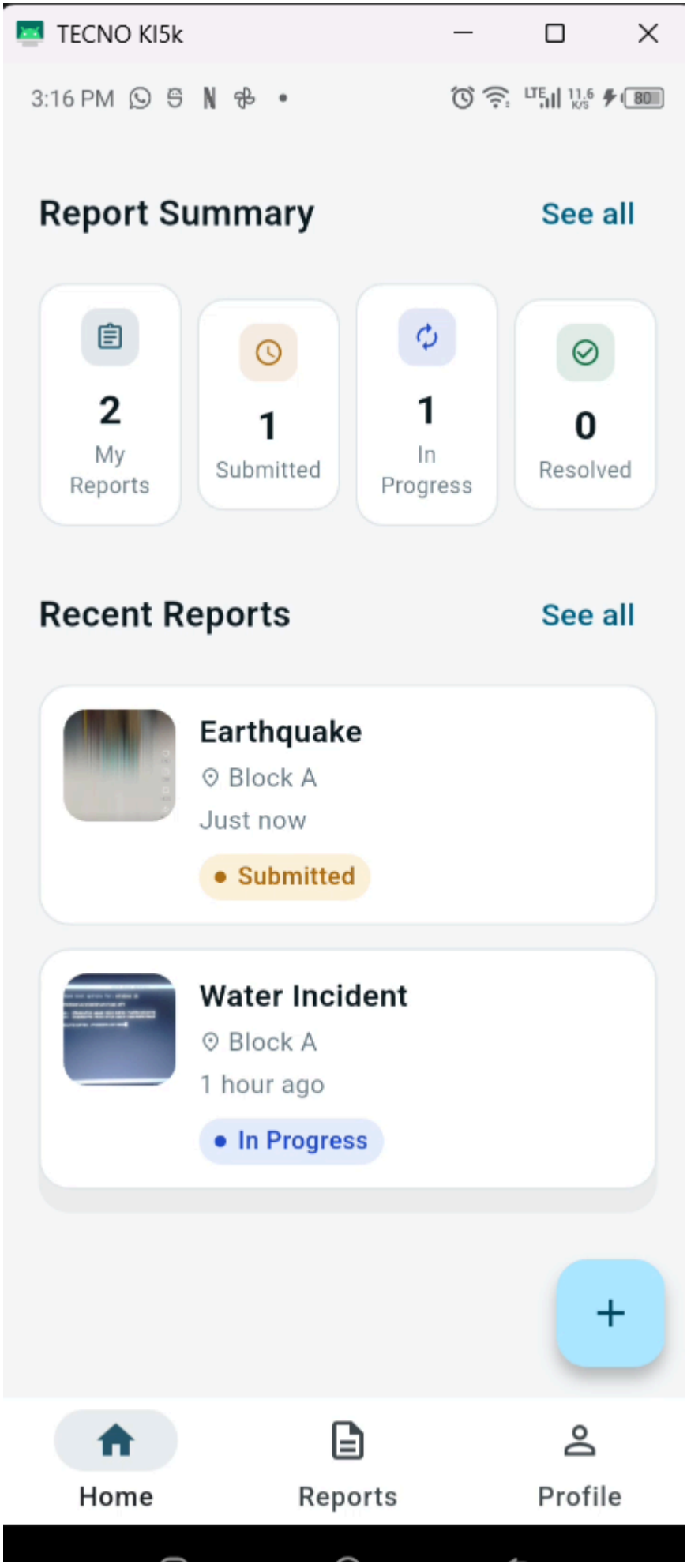
`AuthCubit` owns the question of who is signed in and what role they hold. It exposes a sealed state hierarchy: `AuthInitial`, `AuthLoading`, `AuthAuthenticated`, `AuthUnauthenticated` and `AuthFailure`. Sealed classes were chosen because the compiler then forces every consumer to handle every state, which is how the error state was prevented from being forgotten in the interface.

`ReportCubit` owns the report feed. It subscribes to a Firestore query when a user signs in, stops the subscription when they sign out, and exposes the filter, search term and submission result. The subscription lifecycle matters: with security rules that require an authenticated caller, subscribing before sign-in produces repeated permission errors, and failing to cancel on sign-out leaves a listener attached to a session that no longer exists.

`StaffCubit` owns status transitions. It is small by design, because the transition rule itself lives in `StatusMachine` and the Cubit's job is to check that rule, call the service and report the result.

`setState` was rejected because the state that matters is shared across routes. The signed-in user is needed by the home screen, the report form, the details screen and the profile screen, and passing it through four constructor chains would couple every screen to its parent. Provider would have been a reasonable alternative and is what the same author used on a previous project; Cubit was chosen here because the sealed states make the loading and failure cases explicit in the type system.

Figure 3. The home screen, showing derived counts from the report stream.


TECNO KI5k
3:16 PM
Report Summary
See all
2
My Reports
1
Submitted
1
In Progress
0
Resolved
Recent Reports
See all
Earthquake
Block A
Just now
Submitted
Water Incident
Block A
1 hour ago
In Progress
Home
Reports
Profile



5. User roles and access control

The application has two roles. Students submit reports and read their own. Staff additionally change statuses and see the staff queue.

Two points about this design are important to state plainly.

First, registration always produces a student account. There is no role selector on the sign-up form. A public form that allowed an applicant to declare themselves staff would be a privilege-escalation hole, and the security rules would reject the write in any case. A staff account is created by registering normally and then having an administrator change `role` to `staff` in the `users` document. This is a deliberate operational decision, not a missing feature.

Second, the interface is not the access control. Hiding the staff card and the staff queue from students is a usability decision. The actual enforcement is a server-side rule that runs inside Google's infrastructure, after the client has finished executing, and cannot be bypassed by modifying the application.

The relevant rule reads as follows.

```
function myRole() {
  return get(/databases/$(database)/documents/users/$(request.auth.uid)).data.role;
}

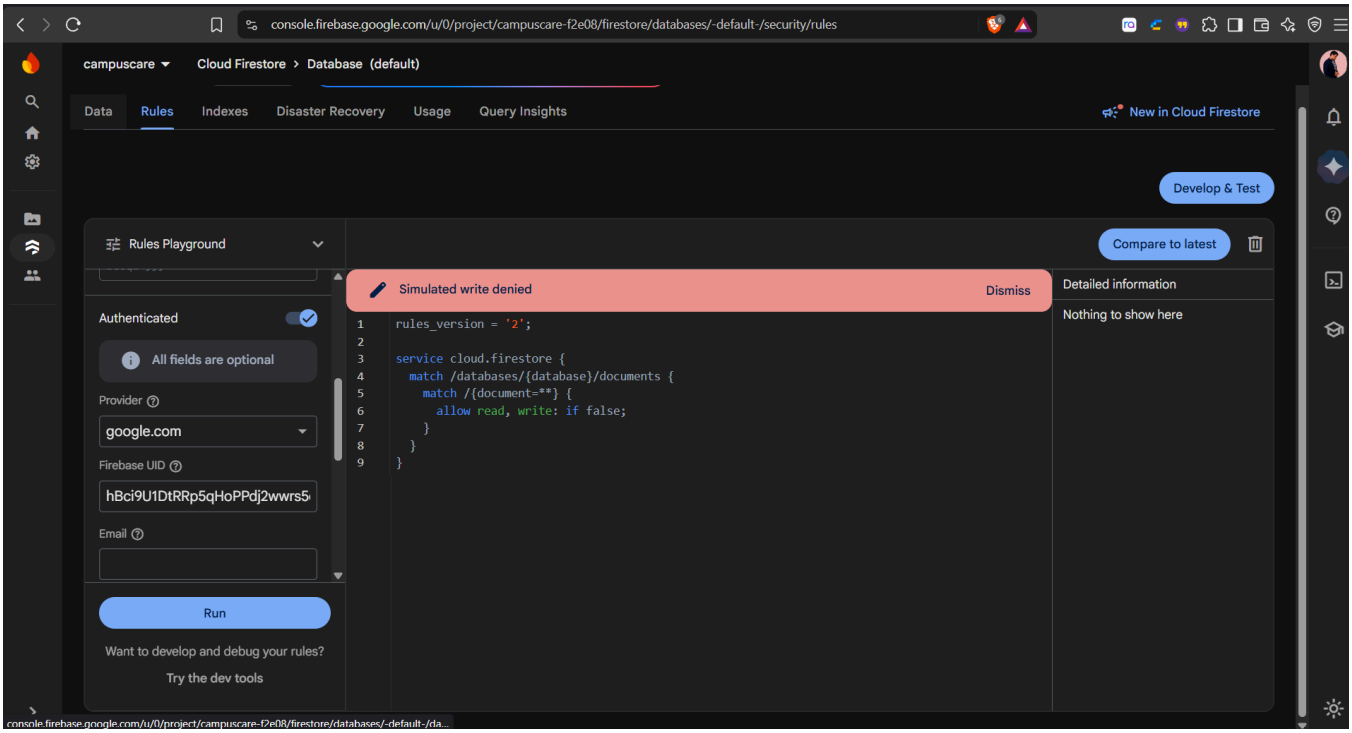
function isStaff() {
  return signedIn() && myRole() == 'staff';
}

match /users/{uid} {
  allow create: if signedIn()
                && request.auth.uid == uid
                && request.resource.data.role == 'student';
  allow update: if signedIn()
                && request.auth.uid == uid
                && request.resource.data.role == resource.data.role;
}

match /reports/{reportId} {
  allow create: if signedIn()
                && request.resource.data.submittedBy.uid == request.auth.uid
                && request.resource.data.status == 'submitted';
  allow update: if isStaff()
                || (signedIn()
                    && resource.data.submittedBy.uid == request.auth.uid
                    && request.resource.data.status == resource.data.status
                    && request.resource.data.submittedBy.uid == resource.data.submittedBy.uid);
}
```

The role is read from the database rather than taken from a value supplied by the client, so a tampered client cannot claim to be staff. The `users` create rule fixes the role to `student`, and the `users` update rule requires the role to be unchanged, which together prevent self-promotion at both points in an account's life. The report update rule permits staff to make any change, while a student may edit their own report's text but not its status or its authorship.

Figure 4. The Rules Playground denying a student's attempt to change a report status.



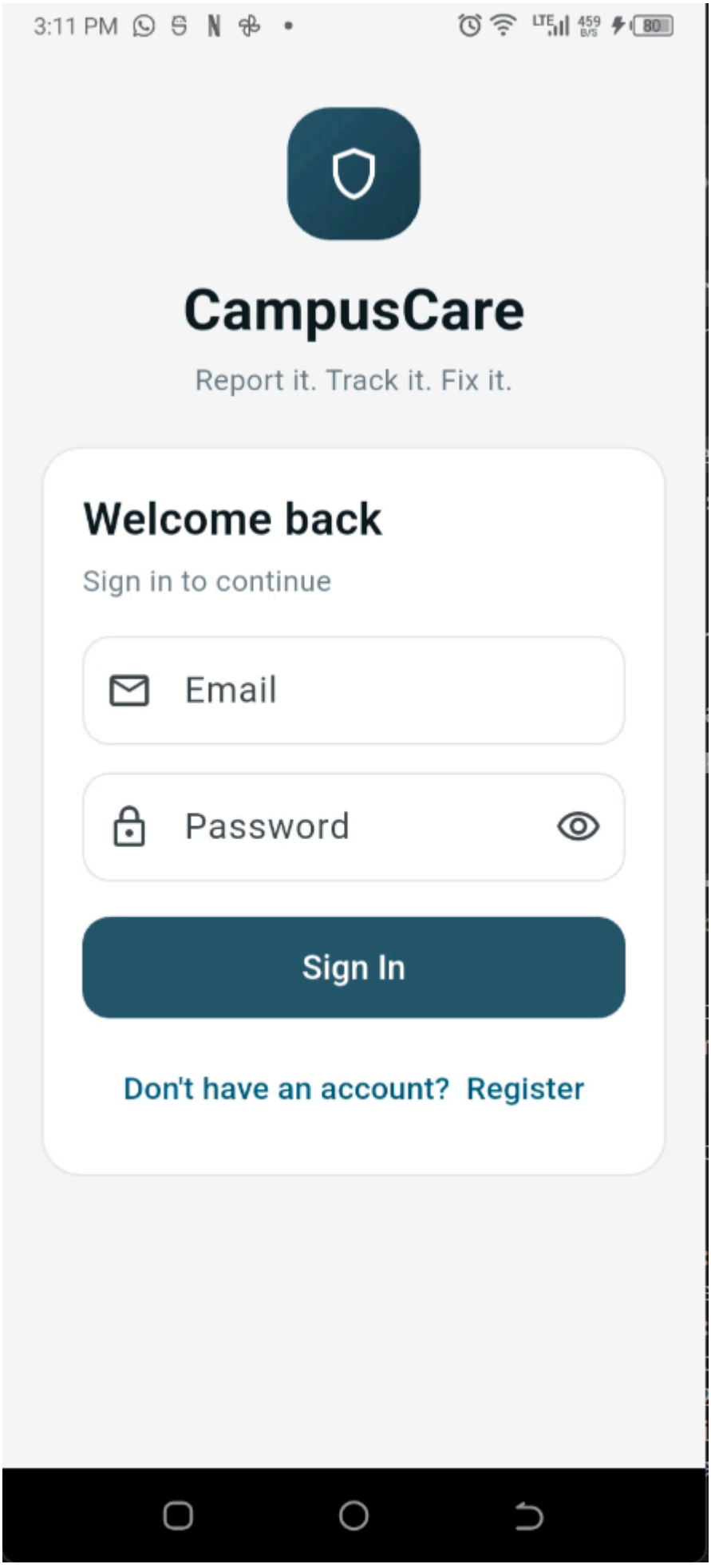
The alternative would have been Firebase custom claims, where the role is stored in the authentication token and read without a document fetch. Custom claims avoid one read per rule evaluation, but they require the Admin SDK to issue them, which means a trusted server that this assignment does not include. The document lookup is the pragmatic choice at this scale, and the trade-off is acknowledged rather than ignored.

One limitation is worth recording. The `users` read rule permits any signed-in user to read any other user's document, which exposes names, email addresses and student numbers to every account holder. For a campus application this should be tightened to allow a user to read only their own document, with a separate public subset for name display. It was left as is to keep the list screens simple within the time available.

---

6. UI and UX decisions

Figure 5. Login and registration.


3:11 PM
CampusCare
Report it. Track it. Fix it.
Welcome back
Sign in to continue
Email
Password
Sign In
Don't have an account? Register

The report form places the category grid above the free-text fields. Category is the field staff triage by, so it is the first decision the user makes rather than the fifth. The five categories use distinct colours that are deliberately separate from the three status colours, so an amber electricity tile cannot be confused with an amber Submitted chip.

Photographs are compressed on the device before upload, using a quality of 70 and a maximum width of 1280 pixels. A four-megabyte phone photograph becomes roughly three hundred kilobytes, which keeps the upload short on a campus network. The form makes the upload timing explicit: after a photo is attached, the caption reads "uploaded when you submit", because the picker itself only reads the file.

Figure 6. The report form with a category selected and a photograph attached.

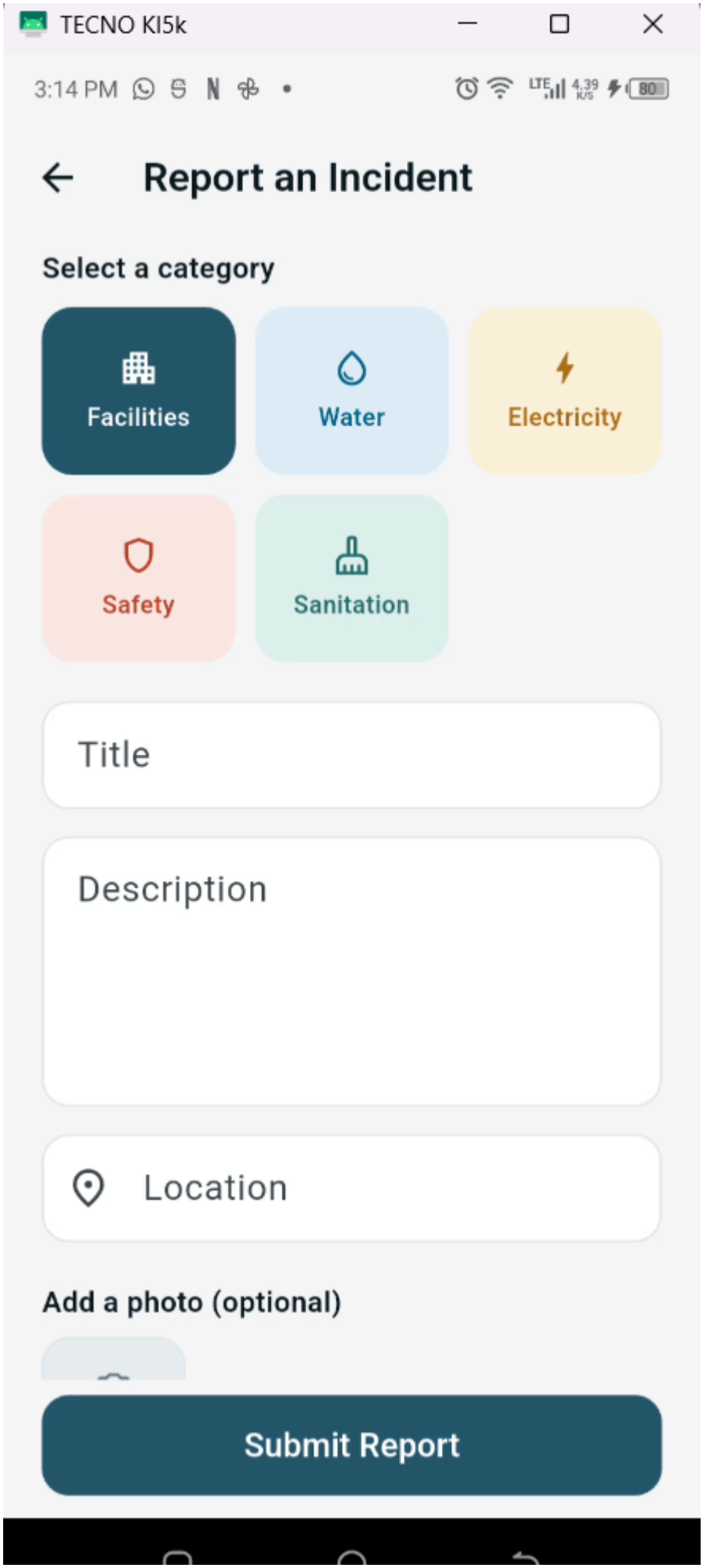

TECNO KI5k
3:14 PM
Report an Incident
Select a category
Facilities
Water
Electricity
Safety
Sanitation
Title
Description
Location
Add a photo (optional)
Submit Report



Figure 7. Confirmation after a successful submission.

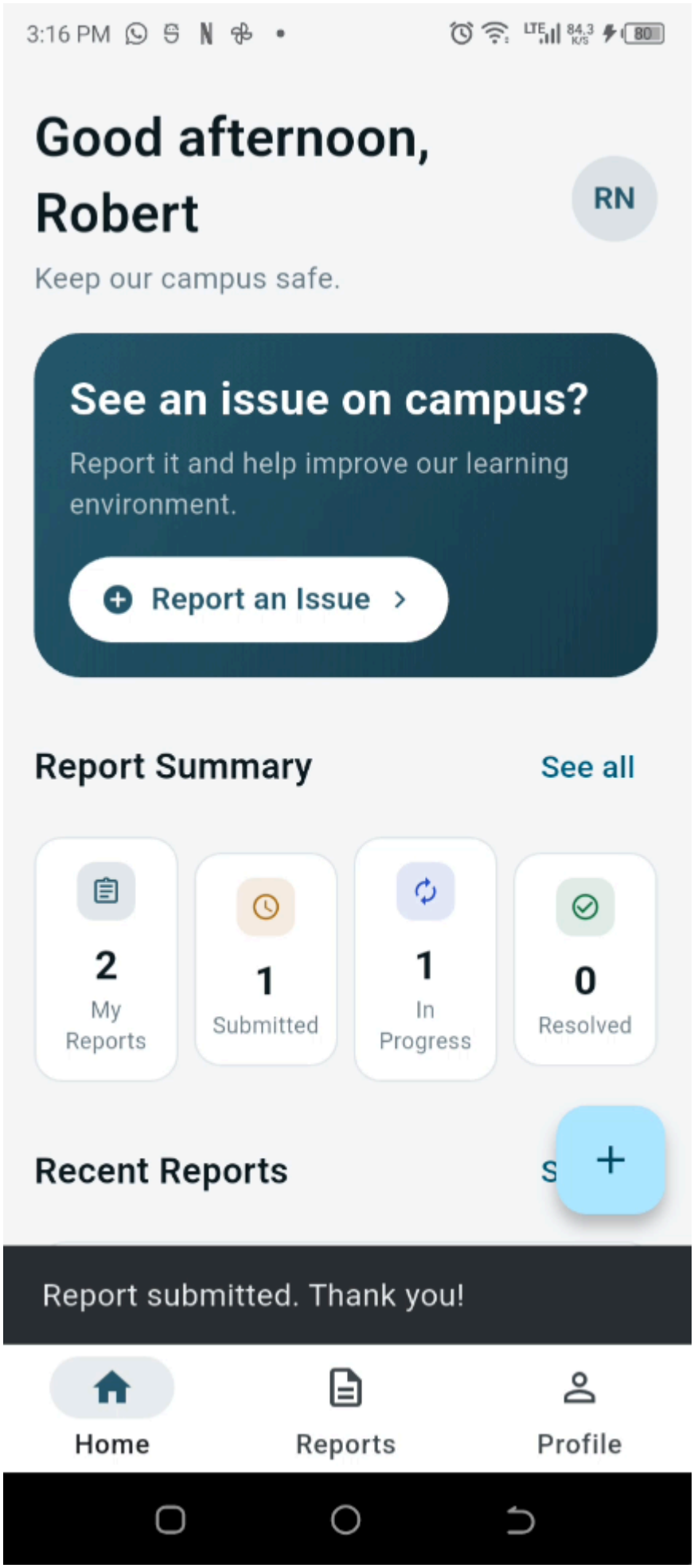
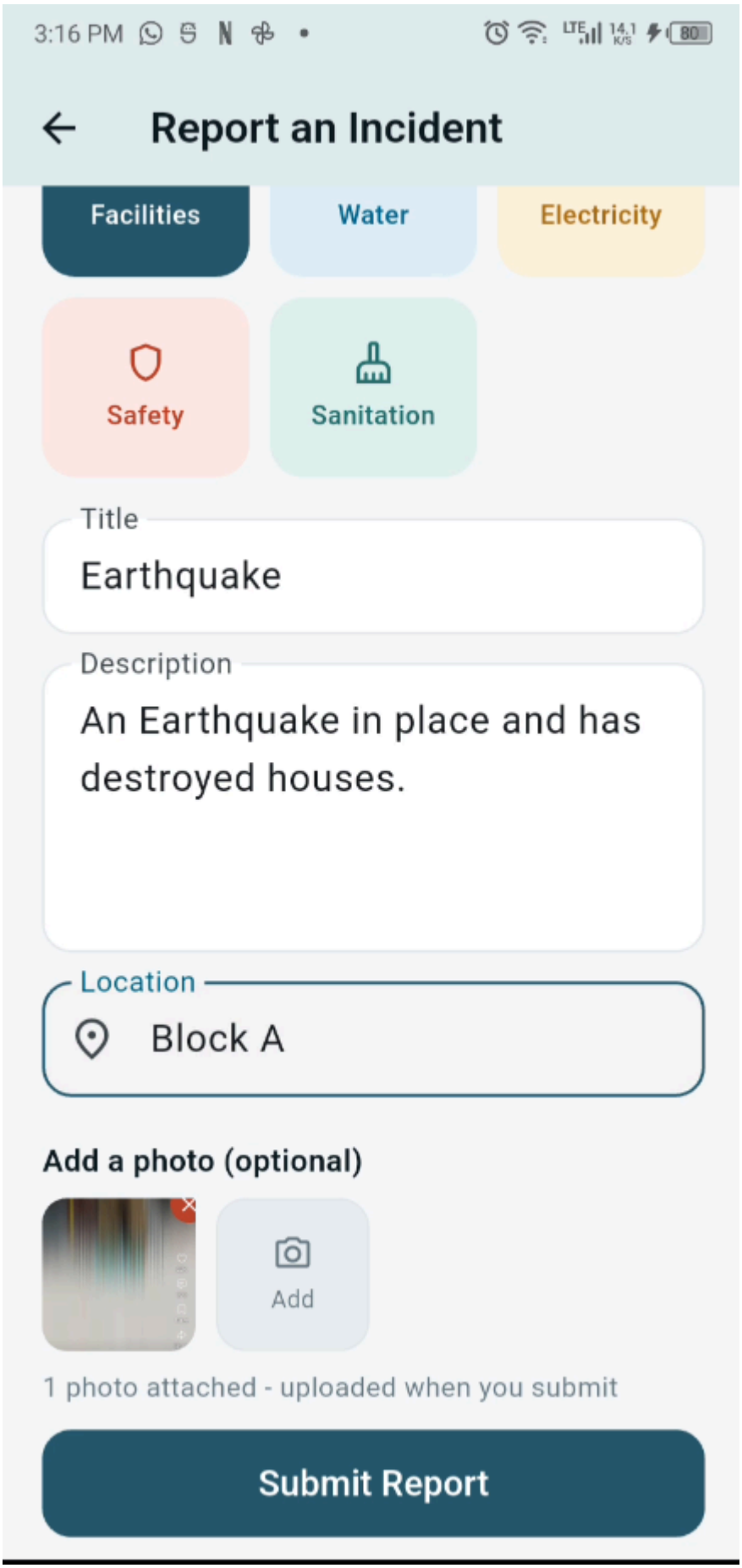

3:16 PM
Good afternoon, Robert
RN
Keep our campus safe.
See an issue on campus?
Report it and help improve our learning environment.
Report an Issue
Report Summary
See all
2
My Reports
1
Submitted
1
In Progress
0
Resolved
Recent Reports
Report submitted. Thank you!
Home
Reports
Profile

Figure 8. A report whose photograph is rendered from the uploaded URL.



Status is presented as a coloured pill in every list, and the report detail screen shows the timeline as a vertical series of entries with a timestamp and an optional note. The effect is that a student can see not only the current state but when it changed and who changed it.

Empty, loading and failure states were implemented rather than assumed. A list with no results shows an explanation and a clear-filters action, not a blank area. A failed submission shows the reason in a snackbar, which turned out to matter for diagnosing the defect described in Section 8.

Figure 9. Status filtering on the report list.

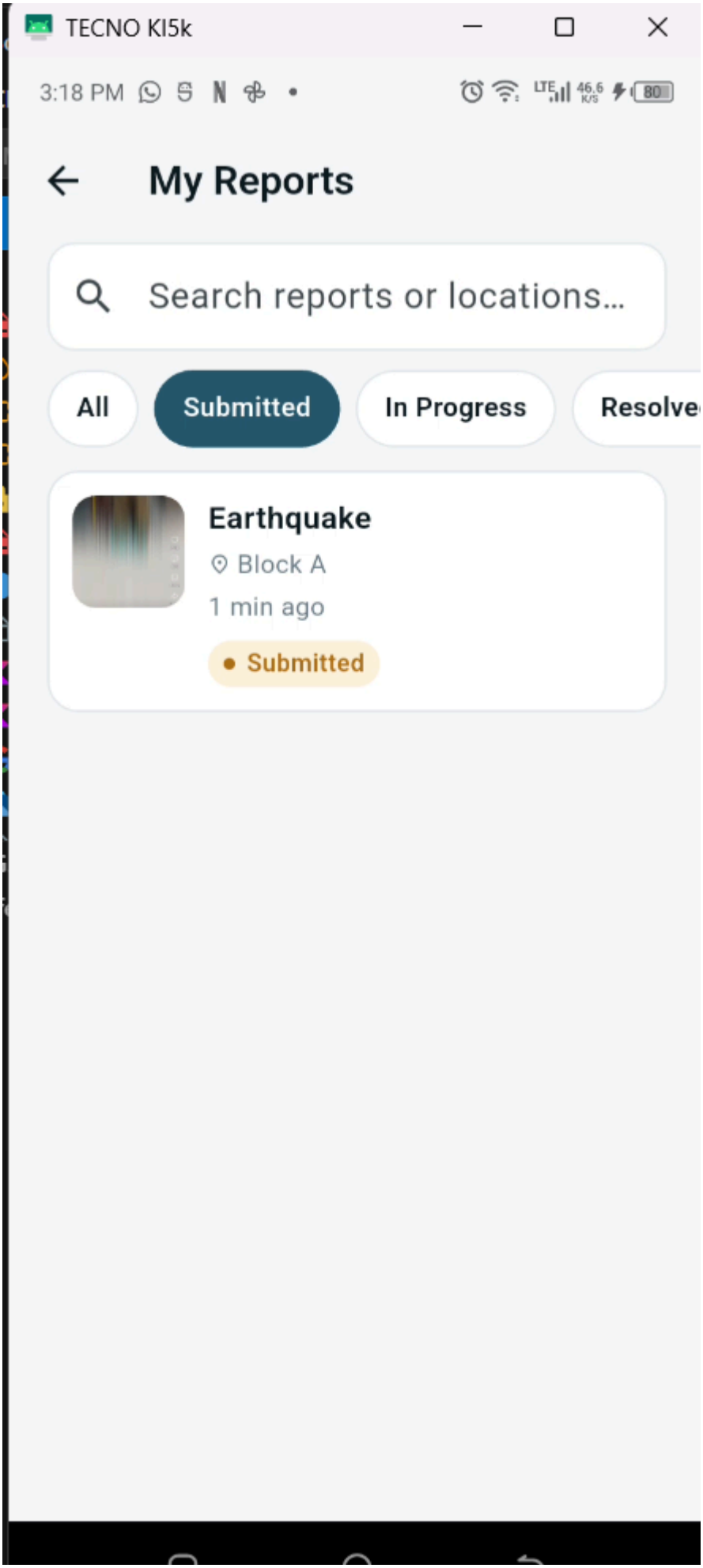
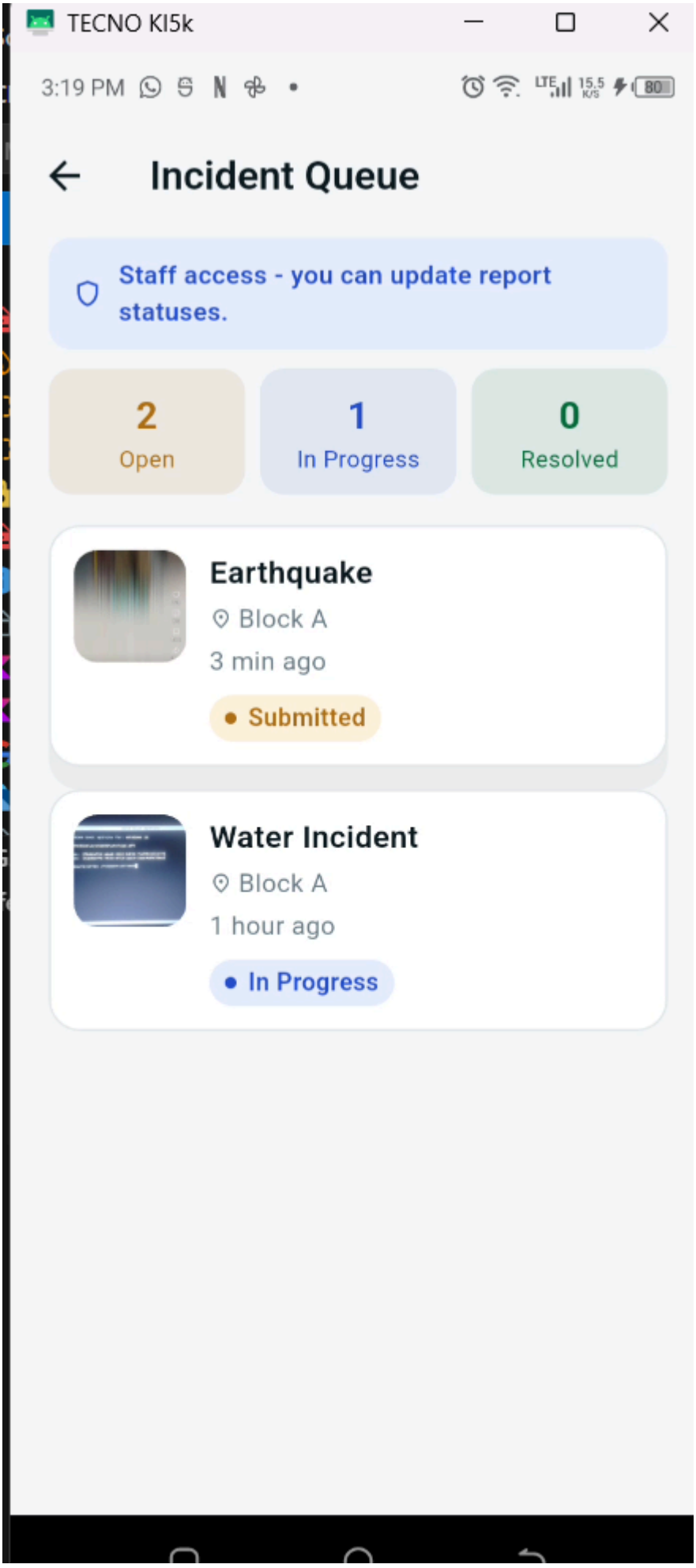

TECNO KI5k
3:18 PM
My Reports
Search reports or locations...
All
Submitted
In Progress
Resolve
Earthquake
Block A
1 min ago
Submitted



Figure 10. The staff queue, visible only to accounts with the staff role.


TECNO KI5k
3:19 PM
Incident Queue
Staff access - you can update report statuses.
2
Open
1
In Progress
0
Resolved
Earthquake
Block A
3 min ago
Submitted
Water Incident
Block A
1 hour ago
In Progress



7. Testing strategy

Testing concentrated on the parts of the application that hold logic rather than layout, because those are the parts that can be tested without a device and the parts most likely to break silently.

Twenty-three unit tests run under `flutter test` and complete in under a second.

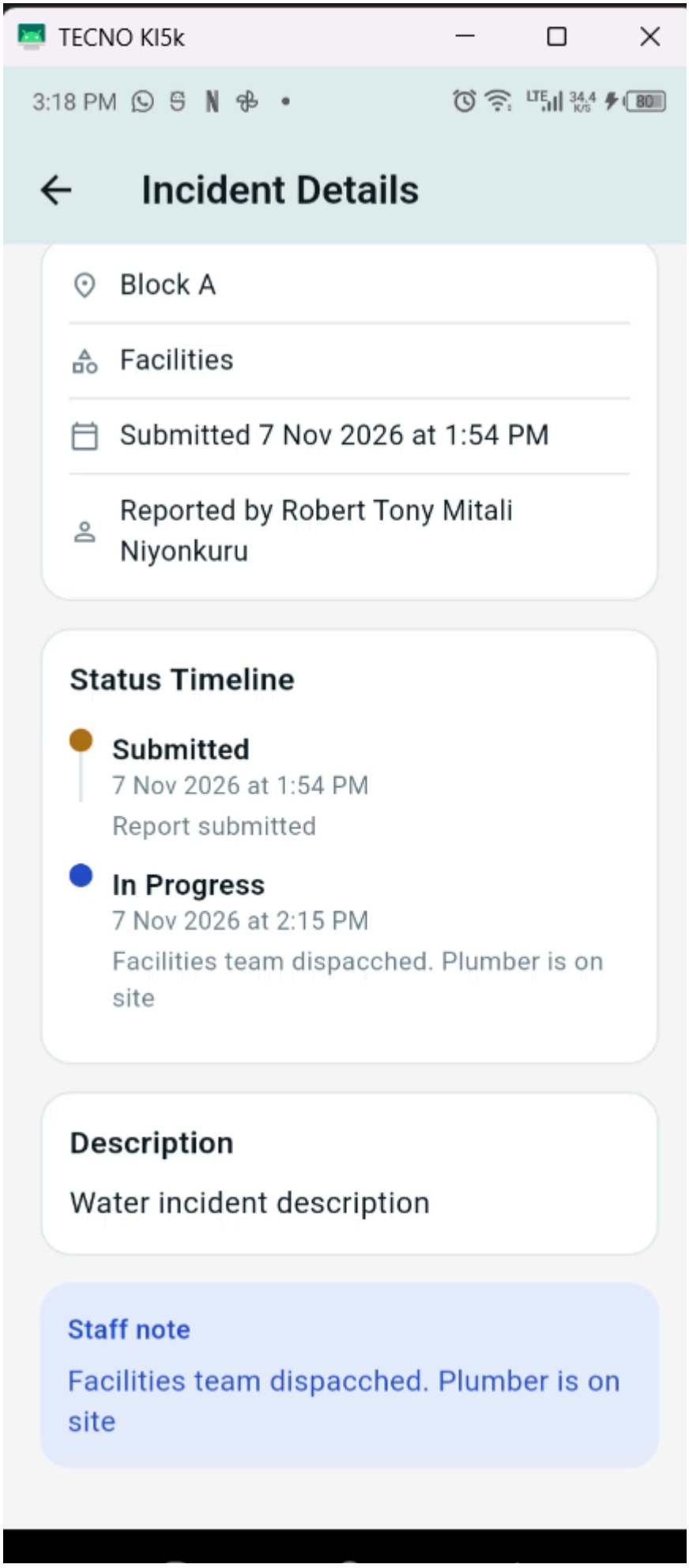
`test/validators_test.dart` covers every form rule: title length bounds, description length bounds, required location, email shape, password length, and the mismatched-password case. These are pure functions with no Firebase dependency, which is why they were written as pure functions.

`test/status_machine_test.dart` covers the transition matrix. It asserts that a submitted report can only move to in progress, that a resolved report reopens only through in progress, and that a self-transition is rejected.

`test/report_test.dart` covers the Firestore payload rather than the interface. It asserts that a new report always carries the submitted status, that the category is written as its Firestore string rather than the Dart enum name, that the reporter is denormalised, and that `createdAt` is a server timestamp sentinel. One test in this file exists specifically to prevent a regression described in Section 8.

Widget tests and rules tests were not implemented. The interface was verified manually on an emulator, and the access-control rules were verified in the Rules Playground. If more time were available, automated tests of the rules using the Firebase emulator suite would be the highest-value addition, because the rules are the part of the system where a mistake is both invisible and consequential.

Figure 11. The report status timeline.


TECNO KI5k
3:18 PM
Incident Details
Block A
Facilities
Submitted 7 Nov 2026 at 1:54 PM
Reported by Robert Tony Mitali Niyonkuru
Status Timeline
Submitted
7 Nov 2026 at 1:54 PM
Report submitted
In Progress
7 Nov 2026 at 2:15 PM
Facilities team dispacched. Plumber is on site
Description
Water incident description
Staff note
Facilities team dispacched. Plumber is on site



8. Challenges encountered

Firestore rejects sentinel values inside arrays. New reports are created with an initial timeline entry. The first implementation wrote `FieldValue.serverTimestamp()` into the `at` field of that entry. Because the entry is an element of an array, Firestore rejected the entire write with an invalid-argument error. Sentinels are only valid as a document field or inside a map, not as an array element. The fix was to write a concrete `Timestamp` for that entry, while the document-level `createdAt` and `updatedAt` continue to use server timestamps. A unit test now asserts that the timeline entry is a `Timestamp` and not a `FieldValue`.

A swallowed exception made that bug invisible. The report Cubit caught the write error, returned a boolean and did not surface the message, and the form only reacted to success. The visible symptom was that the application stayed on the form with no error and no confirmation, while the photograph had already reached the image host. The failure was found by reasoning about the only code path that could produce no reaction at all. The fix was to log the exception, expose it in state, and show it in a snackbar on failure. The general lesson is that a catch block which discards the error converts a loud failure into a silent one.

Firebase Storage requires a paid plan. Firebase Storage was the intended destination for photographs, but new Firebase projects cannot use it on the free tier. The instructor permitted an equivalent cloud storage service, so photographs are uploaded to Cloudinary instead. Because storage had been placed behind a `StorageService` from the start, the substitution required changes to one class and one configuration file, and no screen, model, query or Cubit was touched. The upload uses an unsigned upload preset, which Cloudinary designs for client-side use, so no API secret is shipped in the application binary.

Analyser noise from a generated file. `flutter analyze` reported sixteen issues, all inside a generated plugin registrant. A stray build directory had been copied into the project root, and the analyser was linting generated output. Deleting the directory and adding explicit exclusions for build paths to `analysis_options.yaml` resolved it. The useful part was realising the reported file was not source code.

The Rules Playground simulates the whole resource. When testing the access-control rule, an empty request document produced "write allowed" for a student. This was initially read as a broken rule, but the rule permits a student to edit their own report as long as the status is unchanged, and an empty payload simulates exactly that. The rule was correct; the test was not. The lesson is that a security test must specify the complete request resource, including the field whose change is supposed to be rejected.

A sign-out left a pushed route on the stack. Signing out from the profile screen replaced the root route with the login screen but left the profile route on top of it. Because the signed-in user was now null, the profile screen rendered a progress indicator indefinitely. Pressing the system back button was the only way out. The fix was to pop to the first route after signing out.

---

9. Lessons learned

The first lesson is that access control belongs on the server. Early in the project the staff workflow was understood as a user-interface question: hide the status controls from students. Treating it instead as a rule that the database

enforces changed both the design and the demonstration, because the rule can be tested independently of the application and cannot be defeated by modifying the client.

The second lesson is that a caught exception must be reported. The most expensive defect in this project produced no error message at all, and it survived because the code path that failed returned a value that the interface ignored. Logging the exception was not a debugging convenience; it was the difference between a five-minute diagnosis and an hour of guessing.

A smaller but practical lesson concerns abstraction boundaries. The decision to keep storage behind a single class was made for tidiness rather than foresight. When Firebase Storage turned out to be unavailable, that boundary turned a potential rewrite into a two-file change, which is the clearest argument for the pattern that this project produced.

Future work would add rules tests using the Firebase emulator suite, restrict the `users` read rule, add push notification when a report's status changes, and allow staff to assign a report to themselves so that the queue can be filtered by owner.

References

[1] Google, "Flutter documentation." [Online]. Available: https://docs.flutter.dev. [Accessed: Nov. 7, 2026].

[2] Google, "Cloud Firestore documentation." [Online]. Available: https://firebase.google.com/docs/firestore. [Accessed: Nov. 7, 2026].

[3] Google, "Firebase Authentication documentation." [Online]. Available: https://firebase.google.com/docs/auth. [Accessed: Nov. 7, 2026].

[4] Google, "Security Rules language." [Online]. Available: https://firebase.google.com/docs/rules/rules-language. [Accessed: Nov. 7, 2026].

[5] Bloc, "Cubit, a predictable state management library." [Online]. Available: https://bloclibrary.dev. [Accessed: Nov. 7, 2026].

[6] Cloudinary, "Upload API reference." [Online]. Available: https://cloudinary.com/documentation/image_upload_api_reference. [Accessed: Nov. 7, 2026].